Enhancement Two Narrative: Algorithms and Data Structures

The artifact for this category is an animal shelter dashboard I built for CS 340, Client/Server Development. It was made for a fictional client called Grazioso Salvare, which trains dogs for search and rescue. The dashboard is a Python app built with Dash in a Jupyter notebook, and it pulls animal records from a MongoDB database through a CRUD module I wrote. Users can pick a rescue type, either water, mountain, or disaster, and the dashboard filters the data table to show only dogs that match that type's breed, sex, and age requirements. It also shows a map with the location of the selected animal.

I selected this artifact because the filtering logic was a clear place to improve how the code chooses and uses data. In the original version, each rescue type had its own separate query variable, and the callback function used a chain of if and elif statements to check which radio button was selected and pick the matching query. Every new rescue type would have meant adding another variable and another branch. I replaced all of that with a single dictionary called RESCUE_FILTERS, where the key is the radio button value and the value is the MongoDB query. The callback now picks the query with one line using the dictionary's get() method. If the value is not in the dictionary, like the reset option, it falls back to an empty query that returns all records, which matches what the reset branch did before.

The second change was in the map callback. The original code looked up the animal's latitude, longitude, breed, and name by column number, using calls like iloc[row, 13]. That only works as long as the columns stay in the exact same order. If a column were added or removed from the data, the map would quietly show the wrong information or crash. I changed those lookups to use column names like location_lat and breed, so the map pulls the right data no matter where the column sits.

These changes support the course outcome on designing and evaluating computing solutions using algorithmic principles. A dictionary lookup takes constant time, meaning it does not slow down as more rescue types are added, while an if and elif chain checks each condition one at a time. With only three options the speed difference is tiny, but the bigger benefit is that

the code scales cleanly and is much easier to read and change. The column name change supports the outcome on using well-founded techniques, since naming the data you want is a standard way to make code less fragile.

Working on this made me think about data structures as a design choice and not just a performance choice. The dictionary did not make the dashboard noticeably faster, but it made the code shorter and turned adding a new filter into a one-line change. The iloc change taught me to look for code that only works because of assumptions that are never written down, like column order. The main challenge was making sure the new lookup handled the reset option the same way the old code did, which is why I used the empty query as the default.